

HOCHSCHULE  
HANNOVER  
UNIVERSITY OF  
APPLIED SCIENCES  
AND ARTS

–  
*Fakultät IV  
Wirtschaft und  
Informatik*

# Convolution und ihre Anwendung in der digitalen Audiobearbeitung

*Seminarfach: Audio DSP*

Julian Mueller    Tom Nir  
21. Mai 2026



1 Grundlagen: Audio DSP

2 Convolution

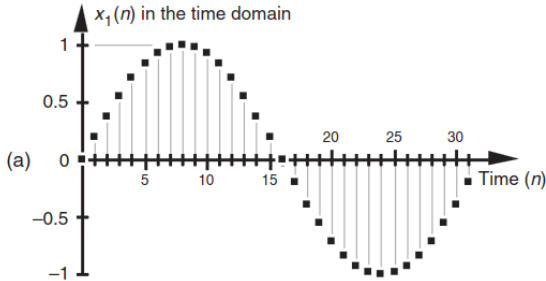
3 Implementierung

4 Literatur



# Sampling I

- Abtastung stetiger Signale  $\Rightarrow$  Folge diskreter Amplitudenwerte



**Abbildung:** Horiz. Achse: Zeitbereich (n-ter Samplewert) Vert. Achse: Amplitude [1, Kap. 2]



# Sampling II

- Berechnung von Signalwerten:

$$x[n] = \sin(2\pi \cdot f_0 \cdot n \cdot t_s) \quad (1)$$

Wobei:

- $2\pi$ : Eine komplette Schwingung ( $360^\circ$  auf dem Einheitskreis)
- $f_0$ : Frequenz des Signals in Hz
- $n$ : Sample-Index
- $f_s$ : Sampling-Intervall (Zeit, die zwischen zwei Messungen vergeht)



# Zeit- und Frequenzdomäne I

- Zeitdomäne: Horizontale -> diskrete Zeitindizes; Vertikale -> Amplitude
- Frequenzdomäne: Horizontale -> Frequenz in  $Hz$ ; Vertikale -> Magnitude

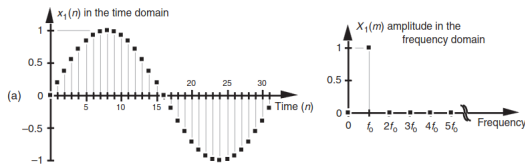


Abbildung: Gegenüberstellung: Zeit-/Frequenzdomäne einer Sinuswelle Lyons, 2011<sup>1</sup>



# Zeit- und Frequenzdomäne II

- Umwandlung eines Signals in Zeitdomäne in Frequenzdomäne mithilfe der *Fourier Transformation*:

$$X[m] = \sum_{n=0}^{N-1} x[n] \cdot e^{\frac{-i2\pi \cdot nm}{N}} \quad (2)$$

---

<sup>1</sup>1, Kap. 2.



# Nyquist-Shannon Grenze und Aliasing I

- Lt. Nyquist-Shannon-Theorem gilt:

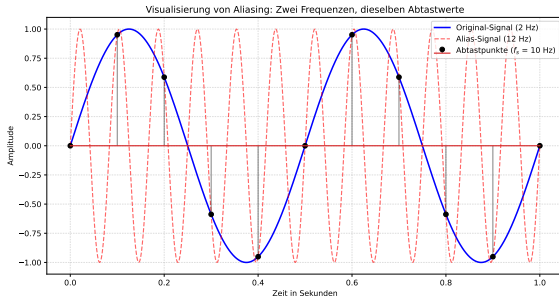
$$x[n] = \sin(2\pi \cdot f_0 \cdot n \cdot t_s) = \sin(2\pi \cdot (f_0 + kf_s) \cdot nt_s) \quad (3)$$

Wobei:  $k \in \mathbb{Z}$

- Daraus Folgt: Die Parameter  $f_0$  und  $(f_0 + kf_s)$  führen zum gleichen Amplitudenwert  $\implies$  Nicht unterscheidbar
- Resultat: Mathematisch unendlich viele Aliase für jede Frequenz des Eingangssignals



# Nyquist-Shannon Grenze und Aliasing II



**Abbildung:** Für jedes Signal gibt es unendlich viele Interpretationsmöglichkeiten

Lyons, 2011<sup>2</sup>

---

<sup>2</sup>1, Kap. 2.





1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



# Laufzeitanalyse I

```
1 Algorithm directConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1      // O(1)
3   for i <- 0 to outLen - 1  // O(N + M + 1)
4     for k <- 0 to M - 1     // O(M)
5       if i-k < 0 or n <= i-k // O(1)
6         continue
7
8       outSignal[i] <- outSignal[i]
9       + irSignal[k] * inSignal[i-k] // Worst-Case: O(1)
10    done
11  done
12  return outSignal
```

Abbildung: Pseudocode: Implementierung der diskreten Convolution



## Laufzeitanalyse II

- Worst-Case Laufzeitklasse:

$$\mathcal{O}(1 + (N + M - 1) \cdot (M + 2)) = \mathcal{O}(M^2 + NM - M + 2N - 1) \quad (4)$$

- ... vereinfacht sich lt. Polynomregel zu:

$$\mathcal{O}(M^2 + NM) \quad (5)$$

- ... es gilt aber

$$N \gg M \implies N \cdot M > M^2 \quad (6)$$

- ... woraus folgt:  $\mathcal{O}(N \cdot M)$



# Laufzeitanalyse III

```
1 Algorithm fftConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1 // O(1)
3   fftLen <- outLen    // O(1)
4
5   // zeroPad() füllt die Listen bis fftLen mit nullen auf
6   zeroPad(inSignal, fftLen) // O(n)
7   zeroPad(irSignal, fftLen) // O(n)
8
9   inSpectrum <- FFT(inSignal, N, fftLen) // O(n log(n))
10  irSpectrum <- FFT(irSignal, M, fftLen) // O(n log(n))
11
12  for f <- 0 to fftLen - 1 // Insg. O(n)
13    outSpectrum[f] <- inSpectrum[f] * irSpectrum[f]
14  done
15
16  outTimeDomain <- IFFT(outSpectrum, fftLen) // O(n log(n))
17
18  // real() isoliert den Realteil des komplexen IFFT-Outputs
19  for i <- 0 to outLen - 1 // Insg. O(n)
20    outSignal[i] <- real(outTime[i])
21  done
22
23  return outSignal
```

**Abbildung:** Pseudocode: Implementierung der Convolution mithilfe der *Fast Fourier Transformation*



# Laufzeitanalyse IV

- Laufzeit:

$$\mathcal{O}(3(N \cdot \log(N)) + 4N + 2) \in \mathcal{O}(N \cdot \log(N)) \quad (7)$$



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



# Direkte Convolution I

```
20  std::vector<float> convolution(std::vector<float>& input, std::vector<float>& ir) {
21
22      unsigned long inputLen {input.size()};
23      unsigned long irLen {ir.size()};
24      unsigned long outputLen {inputLen + irLen - 1};
25
26      std::vector<float> output (outputLen, 0.0f); //-- Mit 0.0f initialisieren
27
28      for (int i = 0; i < outputLen; i++) {
29          for (int k = 0; k < irLen; k++) {
30              if (i - k < 0 || inputLen <= i - k) {
31                  continue;
32              }
33              output[i] += ir[k] * input[i - k];
34          }
35      }
36
37      return output;
38  }
```

Abbildung: Implementierung der direkten Convolution in C++



# Laufzeitmessung I

```
• time ./run
JUCE v8.0.12
Eingangssignal geladen! Anzahl Samples: 416930
Impulsantwort geladen! Anzahl Samples: 176584
Starting convolution ...
[#####-] 99%
Normalisiere Ausgangssignal ...
Output File created successfully!

real    8m55,022s
user    8m54,883s
sys     0m0,024s
```

**Abbildung:** Messung des gezeigten Codes 6 unter Verwendung des Linux-Tools *time*

- Samplerate beider Signale: 44100 Hz
- Eingangssignal: Etwa 10-sekündiges Gitarrensinal
- Impulsantwort: Etwa 5 Sekunden
- Laufzeitklasse:  $\mathcal{O}(N \cdot M)$  (vgl. Folie 10)





# Laufzeitmessung II

- Anzahl der Berechnungen für das  $\approx 15$ -sekündige Ausgangssignal:

$$\begin{aligned} G_{\text{Berechnungen}} &= (N + M - 1) \cdot M \\ &= (416.930 + 176.584 - 1) \cdot 176.584 = 104.804.899.592 \end{aligned} \quad (8)$$

- Entspricht  $\approx 7$ -Milliarden Berechnungen, die der Prozessor in einer Echtzeitanwendung ausführen müsste.



# Fazit I



1 Grundlagen: Audio DSP

2 Convolution

3 Implementierung

4 Literatur



# Literaturverzeichnis I

- [1] Richard G. Lyons. *Understanding Digital Signal Processing*. third. Pearson Education, 2011. ISBN: 978-0-13-702741-5.

